

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	P. Greeshma	Reg. No.:	192424418
Section / Batch:	2024-AI&DS	Date:	01-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Q1 Quick Sort

10 marks

Evaluate Quick Sort's worst-case behavior by executing it on sorted datasets. Record execution time and compare with random input cases. Analyze the degradation to $O(n^2)$. Interpret findings and justify techniques like random pivot selection to avoid worst-case scenarios.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				

Total Marks (100):

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWER:

The screenshot shows the SIMATS Online Programming Lab interface. At the top, there's a header with the SIMATS Engineering logo, the text "SIMATS Online Programming Lab", and "DAA PROGRAMMING". A user profile for "PARVATHA REDDY GREESHMA" is visible on the right. Below the header, there's a section for "Your Code" with a Python implementation of the quicksort algorithm. To the right of the code editor are input and output fields. The input field contains "stdin input". The output field shows the results of the program: a random array, random input time, sorted array, and sorted input time. At the bottom of the code editor, there are buttons for "Run", "Save", and "Submit".

```
1 import time
2 import random
3 def quicksort(arr):
4     if len(arr)<=1:
5         return arr
6     pivot=arr[-1]
7     left=[x for x in arr[:-1]if x<=pivot]
8     right=[x for x in arr[:-1]if x>pivot]
9     return quicksort(left)+[pivot]+quicksort(right)
10 random_arr=random.sample(range(1,100),10)
11 sorted_arr=list(range(1,11))
12 start=time.time()
13 quicksort(random_arr)
14 random_time=time.time()-start
15 start=time.time()
16 quicksort(sorted_arr)
17 sorted_time=time.time()-start
18 print("Random Array:",random_arr)
19 print("Random Input Time:",random_time)
20 print("Sorted Array:",sorted_arr)
21 print("Sorted Input Time:",sorted_time)
```

Output

```
Random Array: [38, 81, 41, 27,
96, 61, 65, 67, 16, 75]
Random Input Time: 0.0
Sorted Array: [1, 2, 3, 4, 5, 6,
7, 8, 9, 10]
Sorted Input Time: 0.0
```

```
import time
import random

def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]
    return quicksort(left) + [pivot] + quicksort(right)

random_arr = random.sample(range(1, 100), 10)
sorted_arr = list(range(1, 11))

start = time.time()
quicksort(random_arr)
random_time = time.time() - start # <-- Corrected

start = time.time()
quicksort(sorted_arr)
```

```
sorted_time = time.time() - start
```

```
print("Random Array:", random_arr)  
print("Random Input Time:", random_time)  
print("Sorted Array:", sorted_arr)  
print("Sorted Input Time:", sorted_time)
```